

Prompt Log — KNN / Schizophrenia Classification

Conversation log reconstructed from the visible chat context. User and AI turns are preserved in chronological order; long code-heavy responses are represented by concise notes where the complete on-screen code was not available as a separately stored transcript.

USER

To apply KNN algorithm on a dataset containing 28 records and around 26 features with the main goal being to classify a target data point
def encode(data) where data is a pandas dataframe
def data_imputation(data) where data is a pandas dataframe
def distance(v1,v2) to calculate euclidean distance between 2 vectors
3 different sorting algorithms: bubble, selection, and merge sort
The KNN flow should identify k nearest neighbours , implement a tie breaking mechanism on distance, and identify the class based on majority

AI

Yep bro ■ this is basically asking you to implement KNN from scratch, including your own preprocessing, distance calculation, sorting, tie-breaking, and majority voting.

The flow is:

Raw data → Encode → Impute missing values → Pick test point → Calculate distances → Sort → Take k nearest → Tie-break if needed → Majority vote → Schizophrenia/Healthy

Here's a clean implementation matching exactly those requirements:

[Full custom KNN implementation was provided, including encode, data_imputation, distance, bubble_sort, selection_sort, merge/merge_sort, and knn.]

USER

in our dataset the label is at the second column where 0 represents healthy and 1 represents schizophrenia.

The features are as follows

subject | label | state0_duration | state0_occurrence | state0_coverage | state1_duration | state1_occurrence | state1_coverage | state2_duration | state2_occurrence | state2_coverage | state3_duration | state3_occurrence | state3_coverage | trans_0to1 | trans_0to2 | trans_0to3 | trans_1to0 | trans_1to2 | trans_1to3 | trans_2to0 | trans_2to1 | trans_2to3 | trans_3to0 | trans_3to1 | trans_3to2

keeping in mind, how does the code change?

AI

YOOO yes — this changes the KNN setup in one very important way. ■

Your columns are:

subject | label | feature1 | feature2 | ... | feature25

So:

- subject → NOT a feature
- label → target
- columns 3–26 → 25 actual features
- 0 → Healthy
- 1 → Schizophrenia

Therefore, we should not use data.iloc[:, :-1] anymore, because that assumes the label is the last column.

The revised code was provided so that `X = data.iloc[:, 2:]` and `y = data.iloc[:, 1]`, with subject excluded from distance calculations. The response

USER

Implement weights in this KNN algorithm to make a weighted KNN classifier if it already doesn't

AI

Nope ■ — the previous version was plain KNN, not weighted KNN.

It used majority voting where every neighbour gets exactly 1 vote. For weighted KNN, closer neighbours get more influence using $\text{weight} = 1 / \text{distance}$.

A revised knn function was provided with weighted voting, zero-distance handling, and distance-based tie-breaking. The response also noted that sklearn's default KNN is unweighted unless `weights='distance'` is specified.

USER

Create a function that uses scikit-learn library to divide the dataset into train and test parts in this format `def A3(data,x,y):` where data is a pandas

AI

Yep bro. Since they specifically want scikit-learn to split the dataset, A3 can use `train_test_split()`.

A version was provided using:

```
from sklearn.model_selection import train_test_split
```

```
def A3(data, x, y):
    X_train, X_test, y_train, y_test = train_test_split(
        data[x], data[y], test_size=0.2, random_state=42, stratify=data[y]
    )
    return X_train, X_test, y_train, y_test
```

The response explained that subject is excluded, label is the target, and stratify is useful for the small 28-record dataset.

USER

Implement a function to train a KNN Classifier with `n_neighbours=3` from the built in scikit learn classifier .

Use the following as reference only

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
neigh = KNeighborsClassifier(n_neighbors=3)
neigh.fit(X_train, y_train)
```

AI

Yep — this one is super straightforward. Since they want the built-in scikit-learn KNN classifier with `n_neighbors=3`:

```
from sklearn.neighbors import KNeighborsClassifier
```

```
def A4(X_train, y_train):
    neigh = KNeighborsClassifier(n_neighbors=3)
    neigh.fit(X_train, y_train)
    return neigh
```

Then:

```
knn_model = A4(X_train, y_train)
y_pred = knn_model.predict(X_test)
```

The response clarified that A4 is specifically the built-in sklearn KNN, separate from the custom KNN.

USER

Now to test the accuracy , implement a function def A5 to use > neigh.score(X_test, y_test) as reference to calculate the required score

AI

Yep! A5 just needs to take the trained KNN model and the test data, then call .score():

```
def A5(neigh, X_test, y_test):
    score = neigh.score(X_test, y_test)
    return score
```

Use it like:

```
score = A5(neigh, X_test, y_test)
print("Accuracy:", score)
print("Accuracy (%):", score * 100)
```

The response explained that neigh.score(X_test, y_test) returns the mean accuracy on the test set.

USER

Define a function A6 that accepts the test vectors and runs a prediction using neigh.predict(Xtest)

AI

Easy ■ A6 just wraps neigh.predict(Xtest):

```
def A6(neigh, Xtest):
    y_pred = neigh.predict(Xtest)
    return y_pred
```

Use it like:

```
y_pred = A6(neigh, X_test)
print("Predicted labels:")
print(y_pred)
```

For this dataset, 0 = Healthy and 1 = Schizophrenia.

USER

Implement the following functions without using sklearn. Complete the functions:

```
def fitit(xtrain,ytrain):
def predictit(xtest,cxtrain,cytrain,k):
def myscore(xtest,ytest,xtrain,ytrain,k):
```

AI

The response supplied custom implementations of fitit, predictit, and myscore without sklearn.

fitit stores copies of the training data.

predictit calculates Euclidean distances, sorts them using merge_sort, takes k neighbours, performs distance-weighted voting using $1/d$, handles zero distance, and uses the closest neighbour for a tie.

myscore calls fitit and predictit, compares predictions with ytest, and returns correct/total.

The response also explained that KNN does not learn coefficients during fit; it essentially stores the training data.

USER

Create a comparative analysis between the function developed by custom and the package function usage. We should do this for a range of values for k and make a plot of accuracies obtained custom developed function and the package function

AI

The response proposed comparing custom KNN and sklearn KNN for k=1 through 10, collecting accuracies and plotting both curves.

It also highlighted an important issue: the custom implementation was distance-weighted while sklearn KNeighborsClassifier is unweighted

USER

instead of creating a function lets just compare the 2 implementations in the main function itself

AI

The response rewrote the comparison directly inside the main function, without creating another comparison function. It used:

```
k_values = range(1, 11)
```

```
custom_scores = []
```

```
sklearn_scores = []
```

For every k it called myscore(...) for the custom implementation and trained KNeighborsClassifier(n_neighbors=k) for the package implementation

USER

Now rewrite the main function to compare the wknn function as well in the mix

AI

The response rewrote main to compare three approaches:

1. Custom normal KNN
2. Custom weighted KNN
3. Scikit-learn KNN

It explained that the existing wknn(data, test_index, k, ...) operates on a single test index, so for the train/test comparison the weighted logic was applied across every test vector inside the main loop. The main collected custom_scores, weighted_scores, and sklearn_scores and plotted all three.

USER

Generate Unit test cases for these functions and return the appropriate main function

AI

The response initially suggested a unittest class, then the user clarified that an OOP structure was not needed. The final direction was to test the custom functions directly in main using assert statements.

The tests covered distance, bubble_sort, selection_sort, merge_sort, encode, data_imputation, fitit, predictit, and myscore. The response also pointed out that the custom predictit implementation was already weighted and that select/dataimputation had implementation issues in the supplied file.

USER

We do not need this OOPS structure. Just test the function for test cases in the main function (return that code)

AI

The response returned a direct main block using assert statements, with test cases for:

- distance with [0,0] and [3,4]
- distance with identical vectors
- bubble sorting
- selection sorting
- merge sorting
- data imputation
- encode
- fitit
- predictit for class 0
- predictit for class 1
- myscore with a small known dataset

It also explained two issues in the supplied code: select() prints rather than returns, and dataimput() calculates the mean of the boolean not-null mask instead of the actual data column.

USER

Now we must generate a prompt log for this entire chat from the first prompt till the last. It should include user and ai responses and must be generated in a PDF format . Export the content to a pdf retaining the conversation log as displayed on screen